

Chương 3.
LẬP TRÌNH SHELL

Mục tiêu

□ Về kiến thức:

- Hiểu khái niệm Shell và Shell Script.
- Biết cách tạo, chỉnh sửa và thực thi Shell Script.
- Biết các câu lệnh cơ bản, các cấu trúc điều kiện, các vòng lặp, hàm.

□ Về kỹ năng:

- Viết được các Shell Script cơ bản để tự động hóa công việc.
- Sử dụng thành thạo các lệnh Shell phổ biến.

□ Tự chủ và trách nhiệm:

- Chủ động tìm hiểu và thực hành Shell Script.
- Tự động hóa công việc bằng các ứng dụng Shell Script

Nội dung giảng dạy

- | | |
|---|----------------------------|
| 1. Giới thiệu về Shell và Shell Script | 9. Toán tử trong Shell |
| 2. Tạo Shell Script | 10. Câu lệnh let |
| 3. Thực thi Shell Script | 11. Câu lệnh bc |
| 4. Biến trong Shell | 12. Câu lệnh awk |
| 5. Nhập giá trị từ bàn phím và lưu vào biến | 13. Cấu trúc điều kiện |
| 6. Hiển thị giá trị ra màn hình | 14. Vòng lặp |
| 7. Chuyển hướng nhập/xuất giá trị | 15. Hàm trong Shell Script |
| 8. Ống dẫn | 16. Bài tập |

Nội dung giảng dạy

- 1. Giới thiệu về Shell và Shell Script**
2. Tạo Shell Script
3. Thực thi Shell Script
4. Biến trong Shell
5. Nhập giá trị từ bàn phím và lưu vào biến
6. Hiển thị giá trị ra màn hình
7. Chuyển hướng nhập/xuất giá trị
8. Ông dẫn
9. Toán tử trong Shell
10. Lệnh let
11. Lệnh bc
12. Lệnh awk
13. Cấu trúc điều kiện
14. Vòng lặp
15. Hàm trong Shell Script
16. Bài tập

3.1. Giới thiệu về Shell và Shell Script

❑ Shell là gì?

- Shell là một chương trình thông dịch lệnh trong Linux/Unix.

❑ Các loại Shell phổ biến:

- Bash (Bourne Again Shell): Phổ biến nhất trên Linux.
- Sh (Bourne Shell): Cổ điển, đơn giản.
- Csh (C Shell): Cú pháp giống ngôn ngữ C.
- Ksh (Korn Shell): Kết hợp ưu điểm của Bourne và C Shell.

❑ Shell Script là gì?

- Shell Script là tập hợp các lệnh được viết trong một tập tin và được thực thi tuần tự.

Nội dung giảng dạy

- | | | | |
|-----------|--|-----|------------------------|
| 1. | Giới thiệu về Shell và Shell Script | 9. | Toán tử trong Shell |
| 2. | Tạo Shell Script | 10. | Lệnh let |
| 3. | Thực thi Shell Script | 11. | Lệnh bc |
| 4. | Biến trong Shell | 12. | Lệnh awk |
| 5. | Nhập giá trị từ bàn phím và lưu vào biến | 13. | Cấu trúc điều kiện |
| 6. | Hiển thị giá trị ra màn hình | 14. | Vòng lặp |
| 7. | Chuyển hướng nhập/xuất giá trị | 15. | Hàm trong Shell Script |
| 8. | Ổng dẫn | 16. | Bài tập |

3.2. Tạo Shell Script

- ❑ **B1:** Tạo tập tin Shell Script bằng các trình soạn thảo nano, vim, gedit, ...
- ❑ **B2:** Gõ lệnh trong tập tin Shell Script vừa tạo.
- ❑ **B3:** Lưu tập tin Shell Script.

❑ Ví dụ 3.1:

- Tạo tập tin shell script “script.sh” bằng nano

```
nano script.sh
```

- Gõ lệnh trong tập tin “script.sh”

```
#!/bin/bash
```

```
echo "Xin chào, đây là shell script đầu tiên!"
```

- Lưu tập tin “script.sh”

```
Ctrl + X, Y, Enter
```

Nội dung giảng dạy

- | | |
|---|----------------------------|
| 1. Giới thiệu về Shell và Shell Script | 9. Toán tử trong Shell |
| 2. Tạo Shell Script | 10. Lệnh let |
| 3. Thực thi Shell Script | 11. Lệnh bc |
| 4. Biến trong Shell | 12. Lệnh awk |
| 5. Nhập giá trị từ bàn phím và lưu vào biến | 13. Cấu trúc điều kiện |
| 6. Hiển thị giá trị ra màn hình | 14. Vòng lặp |
| 7. Chuyển hướng nhập/xuất giá trị | 15. Hàm trong Shell Script |
| 8. Ống dẫn | 16. Bài tập |

3.3. Thực thi Shell Script

- ❑ **B1:** Cấp quyền thực thi cho tập tin Shell Script

```
chmod +x <Tên tập tin Shell Script>
```

- ❑ **B2:** Thực thi tập tin Shell Script (không tham số)

```
./<Tên tập tin Shell Script>
```

- ❑ **Ví dụ 3.2:**

```
chmod +x script.sh  
./script.sh
```

#Kết quả:

Xin chào, đây là shell script đầu tiên!

3.3. Thực thi Shell Script (tt)

- ❑ Có thể thực thi Shell Script cùng các tham số theo cú pháp:

```
./<Tên Shell Script> [$1] [$2] ... [$n]
```

- ❑ Truy xuất các tham số:

- \$0: Tên Shell Script (Tên chương trình)
- \${i} hoặc \$i: Giá trị của tham số i
- \$* hoặc @\$: Danh sách tham số, không tính \$0
- \$#: Số lượng tham số, không tính \$0

3.3. Thực thi Shell Script (tt)

❑ **Ví dụ 3.3:** Tạo và thực thi shell script có nội dung sau:

```
# Hiển thị tên của Shell Script / chương trình
echo "Tên Shell Script / chương trình: $0"

# Hiển thị tổng số tham số (không tính $0)
echo "Số lượng tham số: $#"
```



```
# Hiển thị danh sách tất cả các tham số (không tính $0)
echo "Danh sách tham số (dùng \$*): $*"
```



```
# Duyệt qua từng tham số bằng vòng lặp
echo "Các tham số:"
for (( i=1; i<=$#; i++ )); do
    echo "Tham số $i: ${!i}"
done
```

Nội dung giảng dạy

- | | |
|---|----------------------------|
| 1. Giới thiệu về Shell và Shell Script | 9. Toán tử trong Shell |
| 2. Tạo Shell Script | 10. Lệnh let |
| 3. Thực thi Shell Script | 11. Lệnh bc |
| 4. Biến trong Shell | 12. Lệnh awk |
| 5. Nhập giá trị từ bàn phím và lưu vào biến | 13. Cấu trúc điều kiện |
| 6. Hiển thị giá trị ra màn hình | 14. Vòng lặp |
| 7. Chuyển hướng nhập/xuất giá trị | 15. Hàm trong Shell Script |
| 8. Ống dẫn | 16. Bài tập |

3.4. Biến trong Shell

□ Có 2 loại:

■ Biến hệ thống

- Được tạo và duy trì bởi hệ thống
- Tên biến viết hoa

■ Biến tự tạo

- Được tạo bởi người lập trình
- Tên biến viết thường (tránh nhầm với biến hệ thống)
- Có thể đặt theo qui tắc (tương tự C)

3.4. Biến trong Shell (tt)

❑ Một vài biến hệ thống:

- `$BASH`: Lưu giá trị là tên shell đang dùng
- `$BASH_VERSION`: Lưu giá trị là phiên bản shell đang dùng
- `$HOME`: Lưu giá trị là thư mục cá nhân của người dùng
- `$LOGNAME`: Lưu giá trị là tên người dùng đã đăng nhập
- `$PATH`: Lưu giá trị là đường dẫn đến các thư mục. Các thư mục này sẽ được tìm đến khi chương trình thực thi yêu cầu
- `$PWD`: Lưu giá trị là thư mục hiện hành

3.4. Biến trong Shell (tt)

❑ Biến được tạo bởi người lập trình

- Cú pháp: <Tên biến>=<Giá trị>
- Không có khoảng cách bên trái và bên phải dấu bằng (=)
- Có thể tạo biến có giá trị NULL bằng cách:
<Tên biến>= hoặc <Tên biến>=" "

Nội dung giảng dạy

- | | | | |
|-----------|---|-----|------------------------|
| 1. | Giới thiệu về Shell và Shell Script | 9. | Toán tử trong Shell |
| 2. | Tạo Shell Script | 10. | Lệnh let |
| 3. | Thực thi Shell Script | 11. | Lệnh bc |
| 4. | Biến trong Shell | 12. | Lệnh awk |
| 5. | Nhập giá trị từ bàn phím và lưu vào biến | 13. | Cấu trúc điều kiện |
| 6. | Hiển thị giá trị ra màn hình | 14. | Vòng lặp |
| 7. | Chuyển hướng nhập/xuất giá trị | 15. | Hàm trong Shell Script |
| 8. | Ổng dẫn | 16. | Bài tập |

3.5. Nhập giá trị từ bàn phím và lưu vào biến

❑ Dùng lệnh `read` để đọc giá trị nhập từ bàn phím và lưu vào biến.

❑ **Cú pháp:** `read <biến 1> <biến 2> ... <biến n>`

❑ Ví dụ 3.1

```
#File: read.sh
echo -n "Your first name please: "
read fname
echo "Hello $fname, Lets be friend!"
```

Nội dung giảng dạy

- | | |
|---|----------------------------|
| 1. Giới thiệu về Shell và Shell Script | 9. Toán tử trong Shell |
| 2. Tạo Shell Script | 10. Lệnh let |
| 3. Thực thi Shell Script | 11. Lệnh bc |
| 4. Biến trong Shell | 12. Lệnh awk |
| 5. Nhập giá trị từ bàn phím và lưu vào biến | 13. Cấu trúc điều kiện |
| 6. Hiển thị giá trị ra màn hình | 14. Vòng lặp |
| 7. Chuyển hướng nhập/xuất giá trị | 15. Hàm trong Shell Script |
| 8. Ống dẫn | 16. Bài tập |

3.6. Hiển thị giá trị ra màn hình

- ❑ Dùng lệnh echo để hiển thị chuỗi và xuất giá trị của biến ra màn hình.
- ❑ **Cú pháp:** `echo [các tùy chọn] [chuỗi, ${Tên biến}]`
- ❑ **Các tùy chọn:**
 - `-n`: Không xuống hàng
 - `-e`: Thông dịch các ký tự sau như ký tự đặc biệt
 - `\n`: xuống hàng
 - `\\`: dấu \
 - `\t`: tab ngang
- ❑ **Ví dụ 3.5:** `echo -e "Line 1 \nLine2"`

Nội dung giảng dạy

1. Giới thiệu về Shell và Shell Script
2. Tạo Shell Script
3. Thực thi Shell Script
4. Biến trong Shell
5. Nhập giá trị từ bàn phím và lưu vào biến
6. Hiển thị giá trị ra màn hình
- 7. Chuyển hướng nhập/xuất giá trị**
8. Ống dẫn
9. Toán tử trong Shell
10. Lệnh let
11. Lệnh bc
12. Lệnh awk
13. Cấu trúc điều kiện
14. Vòng lặp
15. Hàm trong Shell Script
16. Bài tập

3.7. Chuyển hướng nhập/xuất giá trị

❑ Bàn phím và màn hình là thiết bị nhập và xuất chuẩn. Ta có thể nhập giá trị từ tập tin và xuất giá trị ra tập tin.

❑ Cú pháp:

- `$command > output_file`: xuất giá trị ra `output_file`
- `$command >> output_file`: nối giá trị xuất ra vào cuối `output_file`
- `$command < input_file`: nhận giá trị từ `input_file`

❑ Ví dụ 3.6:

```
$cat > sname  
red  
green  
blue  
yellow  
Press CTRL + D to save
```

```
$sort < sname > sorted_names  
$cat sorted_names  
blue  
green  
red  
yellow
```

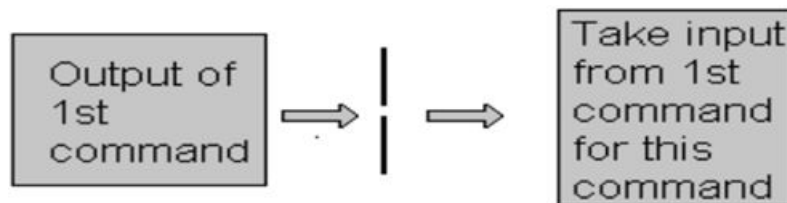
Nội dung giảng dạy

- | | |
|---|----------------------------|
| 1. Giới thiệu về Shell và Shell Script | 9. Toán tử trong Shell |
| 2. Tạo Shell Script | 10. Lệnh let |
| 3. Thực thi Shell Script | 11. Lệnh bc |
| 4. Biến trong Shell | 12. Lệnh awk |
| 5. Nhập giá trị từ bàn phím và lưu vào biến | 13. Cấu trúc điều kiện |
| 6. Hiển thị giá trị ra màn hình | 14. Vòng lặp |
| 7. Chuyển hướng nhập/xuất giá trị | 15. Hàm trong Shell Script |
| 8. Ống dẫn | 16. Bài tập |

3.8. Ống dẫn

- ❑ Ống dẫn là một kỹ thuật chuyển kết xuất của một chương trình vào đối số nhập của một chương trình khác.

- ❑ **Cú pháp:** `command1 | command2`



3.8. Ống dẫn (tt)

❏ Ví dụ 3.7:

- Hiển thị kết quả lệnh ls từng trang màn hình

```
ls | more
```

- Đếm và hiển thị số lượng thư mục có trong thư mục hiện hành

```
ls -l | grep ^d | wc -l #Không đếm thư mục ẩn
```

#Đếm cả thư mục ẩn

```
ls -l | find . -type d -maxdepth 1 | wc -l
```

```
ls -l | find . -type d -maxdepth 1 | tail -n +2 | wc -l
```


3.8. Ống dẫn (tt)

❏ Ví dụ 3.7: (tiếp)

- Đếm và hiển thị số lượng tập tin có trong thư mục hiện hành

```
ls -l | grep ^- | wc -l #Không đếm tập tin ẩn
```

```
#Đếm cả tập tin ẩn
```

```
ls -l | find . -type f -maxdepth 1 | wc -l
```

- Trích dữ liệu từ dòng 20 đến dòng 30 của tập tin hotel.txt và lưu vào tập tin hlist

```
head -30 < hotel.txt | tail -10 > hlist
```

- Lấy những dòng không trùng lặp từ tập tin sname và lưu vào tập tin u_sname

```
sort < sname | uniq > u_sname
```

Nội dung giảng dạy

- | | |
|---|-------------------------------|
| 1. Giới thiệu về Shell và Shell Script | 9. Toán tử trong Shell |
| 2. Tạo Shell Script | 10. Lệnh let |
| 3. Thực thi Shell Script | 11. Lệnh bc |
| 4. Biến trong Shell | 12. Lệnh awk |
| 5. Nhập giá trị từ bàn phím và lưu vào biến | 13. Cấu trúc điều kiện |
| 6. Hiển thị giá trị ra màn hình | 14. Vòng lặp |
| 7. Chuyển hướng nhập/xuất giá trị | 15. Hàm trong Shell Script |
| 8. Ống dẫn | 16. Bài tập |

3.9. Toán tử trong Shell

❑ Toán tử số học dùng để thực hiện các phép toán cơ bản như cộng, trừ, nhân, chia.

❑ **Các toán tử số học:**

▪ Cộng: +

▪ Trừ: -

▪ Nhân: *

▪ Chia: /

▪ Lấy dư: %

▪ Lũy thừa: ** (trong bash 4.0 trở lên)

Ví dụ 3.8:

```
a=5
```

```
b=3
```

```
sum=$((a + b))
```

```
echo "Tổng: $sum"
```

3.9. Toán tử trong Shell (tt)

❑ Toán tử so sánh dùng để so sánh hai giá trị số.

❑ **Các toán tử so sánh:**

- Bằng: `-eq`
- Khác: `-ne`
- Lớn hơn: `-gt`
- Nhỏ hơn: `-lt`
- Lớn hơn hoặc bằng: `-ge`
- Nhỏ hơn hoặc bằng: `-le`

Ví dụ 3.9:

```
if [ $a -eq $b ]; then  
    echo "Hai số bằng nhau"  
fi
```

3.9. Toán tử trong Shell (tt)

❑ Toán tử trên chuỗi dùng để so sánh các chuỗi ký tự.

❑ Các toán tử trên chuỗi:

- Bằng: =
- Khác: !=
- Rỗng: -z (true nếu chuỗi rỗng)
- Không rỗng: -n (true nếu chuỗi không rỗng)

Ví dụ 3.10:

```
str1="Hello"  
str2="World"  
if [ "$str1" = "$str2" ]; then  
    echo "Chuỗi bằng nhau"  
else  
    echo "Chuỗi khác nhau"  
fi
```

3.9. Toán tử trong Shell (tt)

❑ Toán tử trên logic dùng để kết hợp nhiều điều kiện.

❑ Các toán tử logic:

- Và: `&&` hoặc `-a`
- Hoặc: `||` hoặc `-o`
- Phủ định: `!`

Ví dụ 3.11:

```
if [ $a -gt 0 ] && [ $b -gt 0 ]; then  
    echo "Cả hai số đều dương"  
fi
```

3.9. Toán tử trong Shell (tt)

❑ Toán tử trên tập tin dùng để kiểm tra trạng thái của tập tin.

❑ **Các toán tử tập tin:**

- -f: Kiểm tra tập tin thường
- -d: Kiểm tra thư mục
- -e: Kiểm tra tập tin tồn tại
- -r: Kiểm tra quyền đọc
- -w: Kiểm tra quyền ghi
- -x: Kiểm tra quyền thực thi
- -s: Kiểm tra tập tin không rỗng

Ví dụ 3.12:

```
if [ -f "/etc/passwd" ]; then  
    echo "Là tập tin"  
fi
```

Nội dung giảng dạy

- | | |
|---|----------------------------|
| 1. Giới thiệu về Shell và Shell Script | 9. Toán tử trong Shell |
| 2. Tạo Shell Script | 10. Lệnh let |
| 3. Thực thi Shell Script | 11. Lệnh bc |
| 4. Biến trong Shell | 12. Lệnh awk |
| 5. Nhập giá trị từ bàn phím và lưu vào biến | 13. Cấu trúc điều kiện |
| 6. Hiển thị giá trị ra màn hình | 14. Vòng lặp |
| 7. Chuyển hướng nhập/xuất giá trị | 15. Hàm trong Shell Script |
| 8. Ống dẫn | 16. Bài tập |

3.10. Lệnh let

❑ **let** là lệnh dùng để thực hiện các phép toán số nguyên.

❑ **Cú pháp:** `let <biểu thức>`

❑ **Ví dụ 3.13:**

- Cộng (+): `let sum=5+3`
- Trừ (-): `let diff=10-7`
- Nhân (*): `let prod=4*5`
- Chia (/): `let quot=20/4`
- Lũy thừa (**): `let pow=2**3`
- Tăng, giảm:
 - Tăng 1: `let count++`
 - Giảm 1: `let count--`

3.10. Lệnh let (tt)

❑ Một số cách viết tương đương lệnh let:

- Dùng `$ ( ( ... ) )`:

Ví dụ 3.14:

```
sum=$(( $1 + $2 ))  
echo "Tổng là: $sum"
```

- Dùng `expr`:

Ví dụ 3.15:

```
sum=$(expr $1 + $2)  
echo "Tổng là: $sum"
```

Nội dung giảng dạy

- | | |
|---|----------------------------|
| 1. Giới thiệu về Shell và Shell Script | 9. Toán tử trong Shell |
| 2. Tạo Shell Script | 10. Lệnh let |
| 3. Thực thi Shell Script | 11. Lệnh bc |
| 4. Biến trong Shell | 12. Lệnh awk |
| 5. Nhập giá trị từ bàn phím và lưu vào biến | 13. Cấu trúc điều kiện |
| 6. Hiển thị giá trị ra màn hình | 14. Vòng lặp |
| 7. Chuyển hướng nhập/xuất giá trị | 15. Hàm trong Shell Script |
| 8. Ống dẫn | 16. Bài tập |

3.11. Lệnh bc

❑ **bc** là lệnh để tính toán với số thực.

❑ **Cú pháp:** `echo "biểu_thức" | bc`

❑ **Ví dụ 3.16:**

```
#Kết hợp với scale để xác định số chữ số thập phân
echo "scale=2; 5 / 3" | bc #Chia làm tròn đến 2 chữ số thập
phân
echo "scale=3; 5^3" | bc      # Lũy thừa
echo "scale=4; sqrt(16)" | bc # Căn bậc hai

#Biến trong Shell kết hợp với bc
num1=10.5
num2=3.7
result=$(echo "scale=2; $num1 + $num2" | bc)
echo "Kết quả cộng: $result"
```

Nội dung giảng dạy

- | | |
|---|----------------------------|
| 1. Giới thiệu về Shell và Shell Script | 9. Toán tử trong Shell |
| 2. Tạo Shell Script | 10. Lệnh let |
| 3. Thực thi Shell Script | 11. Lệnh bc |
| 4. Biến trong Shell | 12. Lệnh awk |
| 5. Nhập giá trị từ bàn phím và lưu vào biến | 13. Cấu trúc điều kiện |
| 6. Hiển thị giá trị ra màn hình | 14. Vòng lặp |
| 7. Chuyển hướng nhập/xuất giá trị | 15. Hàm trong Shell Script |
| 8. Ống dẫn | 16. Bài tập |

3.12. Lệnh awk

- ❑ **awk** là lệnh để xử lý văn bản và tính toán.
- ❑ **Cú pháp:** `awk 'biểu_thức' file.txt`
- ❑ **Cú pháp phổ biến:**
 - `$1, $2, ...`: Đại diện cho các cột trong dòng.
 - `{print $1, $2}`: In ra cột 1 và cột 2.
 - `-F`: Đặt dấu phân tách (thường là dấu phẩy hoặc khoảng trắng).

3.12. Lệnh awk (tt)

❏ **Ví dụ 3.17:** Tính tổng của một cột trong file.

```
#Giả sử file data.txt có nội dung:
```

```
10
```

```
20
```

```
30
```

```
40
```

```
#Tính tổng của cột một trong file data.txt
```

```
awk '{sum += $1} END {print "Tổng:", sum}' data.txt
```

```
#K ết  quả
```

```
Tổng: 100
```

3.12. Lệnh awk (tt)

❏ Ví dụ 3.18: Lọc dữ liệu từ file CSV

```
#Giả sử file scores.csv có nội dung:
```

```
Alice,85
```

```
Bob,90
```

```
Charlie,78
```

```
David,92
```

```
#Lọc các dòng dữ liệu có cột hai >= 80 từ file scores.csv
```

```
awk -F, '$2 >= 80 {print $1, $2}' scores.csv
```

```
#Kết quả
```

```
Alice 85
```

```
Bob 90
```

```
David 92
```


3.12. Lệnh awk (tt)

❏ Ví dụ 3.19: Tính trung bình từ file

```
#Giả sử file scores.csv có nội dung:
```

```
Alice,85
```

```
Bob,90
```

```
Charlie,78
```

```
David,92
```

```
#Tính trung bình từ file scores.csv
```

```
awk -F, '{sum+=$2; count++} END {print "Trung bình:", sum/count}' scores.csv
```

```
#Kết quả
```

```
Trung bình: 86.25
```

So sánh `let`, `bc`, `awk`

Lệnh	Sử dụng	Đặc điểm
<code>let</code>	Phép toán số nguyên	Nhanh, đơn giản, không hỗ trợ số thực
<code>bc</code>	Phép toán số thực và phức tạp	Hỗ trợ làm tròn và tính lũy thừa
<code>awk</code>	Xử lý văn bản và tính toán	Hỗ trợ tốt dữ liệu dạng bảng (CSV, TSV)

Nội dung giảng dạy

- | | |
|---|-------------------------------|
| 1. Giới thiệu về Shell và Shell Script | 9. Toán tử trong Shell |
| 2. Tạo Shell Script | 10. Lệnh let |
| 3. Thực thi Shell Script | 11. Lệnh bc |
| 4. Biến trong Shell | 12. Lệnh awk |
| 5. Nhập giá trị từ bàn phím và lưu vào biến | 13. Cấu trúc điều kiện |
| 6. Hiển thị giá trị ra màn hình | 14. Vòng lặp |
| 7. Chuyển hướng nhập/xuất giá trị | 15. Hàm trong Shell Script |
| 8. Ống dẫn | 16. Bài tập |

3.13. Cấu trúc điều kiện

❑ Cấu trúc `if-then` dùng để kiểm tra một điều kiện đơn giản.

❑ **Cú pháp:**

```
if [ đi_ề_u_kiệ_n ];  
then  
    lệ_nh  
fi
```

❑ **Ví dụ 3.20:**

```
a=5  
b=3  
if [ $a -gt $b ]; then  
    echo "$a lớn hơn $b"  
fi
```

3.13. Cấu trúc điều kiện (tt)

- ❑ Cấu trúc `if-else` dùng khi cần thực hiện một khối lệnh khác nếu điều kiện sai.
- ❑ **Cú pháp:**

```
if [ đi_ề_u_kiệ_n ];  
then  
    lệ_nh_1  
else  
    lệ_nh_2  
fi
```

Ví dụ 3.21:

```
if [ $a -eq $b ]; then  
    echo "Hai số bằng nhau"  
else  
    echo "Hai số khác nhau"  
fi
```

3.13. Cấu trúc điều kiện (tt)

❑ Cấu trúc `if-elif-else` dùng khi có nhiều điều kiện cần kiểm tra.

❑ **Cú pháp:**

```
if [ đi_ề_u_kiệ_n_1 ]; then
    lện_h_1
elif [ đi_ề_u_kiệ_n_2 ];
then
    lện_h_2
else
    lện_h_mặ_c_định
fi
```

Ví dụ 3.22:

```
if [ $a -gt $b ]; then
    echo "$a lớn hơn $b"
elif [ $a -lt $b ]; then
    echo "$a nhỏ hơn $b"
else
    echo "$a bằng $b"
fi
```

3.13. Cấu trúc điều kiện (tt)

- ❑ Cấu trúc **case** dùng khi có nhiều lựa chọn và mỗi lựa chọn tương ứng với một khối lệnh.

- ❑ **Cú pháp:**

```
case $biến in
    "giá_trị_1")
        lệnh_1
        ;;
    "giá_trị_2")
        lệnh_2
        ;;
    *)
        lệnh_mặc_định
        ;;
esac
```

Ví dụ 3.23:

```
echo "Nhập hệ điều hành:"
read os
case $os in
    "Ubuntu")
        echo "Bạn đang dùng Ubuntu";;
    "Debian")
        echo "Bạn đang dùng Debian";;
    "Fedora")
        echo "Bạn đang dùng Fedora";;
    *)
        echo "Hệ điều hành không xác định";;
esac
```

Nội dung giảng dạy

- | | |
|---|----------------------------|
| 1. Giới thiệu về Shell và Shell Script | 9. Toán tử trong Shell |
| 2. Tạo Shell Script | 10. Lệnh let |
| 3. Thực thi Shell Script | 11. Lệnh bc |
| 4. Biến trong Shell | 12. Lệnh awk |
| 5. Nhập giá trị từ bàn phím và lưu vào biến | 13. Cấu trúc điều kiện |
| 6. Hiển thị giá trị ra màn hình | 14. Vòng lặp |
| 7. Chuyển hướng nhập/xuất giá trị | 15. Hàm trong Shell Script |
| 8. Ống dẫn | 16. Bài tập |

3.14. Vòng lặp

❑ Vòng lặp `for` dùng để lặp qua một tập hợp các giá trị.

❑ **Cú pháp 1:**

```
for variable in list
do
    lệnh
done
```

❑ **Cú pháp 2:**

```
for ((expr1; expr2; expr3))
do
    Lặp tất cả các lệnh giữa do và done trong khi expr2 đúng
done
```

3.14. Vòng lặp (tt)

- ❑ Vòng lặp `for` dùng để lặp qua một tập hợp các giá trị.

Ví dụ 3.24:

```
for i in 1 2 3 4 5
do
    echo "Số: $i"
done
```

Ví dụ 3.25: Vòng lặp for với mảng

```
arr=("Ubuntu" "Debian" "Fedora")
for os in "${arr[@]}"
do
    echo "Hệ điều hành: $os"
done
```

3.14. Vòng lặp (tt)

❑ Vòng lặp `while` lặp lại một khối lệnh khi điều kiện còn đúng.

❑ **Cú pháp:**

```
while [ đi_ề_u_ki_ệ_n
]
do
    l_ệ_n_h
done
```

Ví dụ 3.26:

```
count=1
while [ $count -le 5 ]
do
    echo "Lần lặ_p: $count"
    ((count++))
done
```

3.14. Vòng lặp (tt)

- ❑ Vòng lặp `until` ngược với `while`, lặp lại khi điều kiện **sai**.
- ❑ Cú pháp:

```
until [ đi_ề_u_ki_ệ_n  
]  
do  
    l_ệ_n_h  
done
```

Ví dụ 3.27:

```
count=1  
until [ $count -gt 5 ]  
do  
    echo "Lần lặp: $count"  
    ((count++))  
done
```

3.14. Vòng lặp (tt)

❑ Kết hợp nhiều vòng lặp với nhau.

❑ **Ví dụ 3.28:**

```
for i in 1 2 3
do
    for j in A B C
    do
        echo "Vòng ngoài: $i, Vòng trong: $j"
    done
done
```

3.14. Vòng lặp (tt)

❑ Câu lệnh **break** thoát khỏi vòng lặp.

❑ **Ví dụ 3.29:**

```
for i in 1 2 3 4 5
do
    if [ $i -eq 3 ]; then
        break
    fi
    echo "Số: $i"
done
```

3.14. Vòng lặp (tt)

- ❑ Câu lệnh `continue` bỏ qua lần lặp hiện tại và chuyển sang lần lặp tiếp theo.
- ❑ Ví dụ 3.30:

```
for i in 1 2 3 4 5
do
    if [ $i -eq 3 ]; then
        continue
    fi
    echo "Số: $i"
done
```

Nội dung giảng dạy

- | | |
|---|-----------------------------------|
| 1. Giới thiệu về Shell và Shell Script | 9. Toán tử trong Shell |
| 2. Tạo Shell Script | 10. Lệnh let |
| 3. Thực thi Shell Script | 11. Lệnh bc |
| 4. Biến trong Shell | 12. Lệnh awk |
| 5. Nhập giá trị từ bàn phím và lưu vào biến | 13. Cấu trúc điều kiện |
| 6. Hiển thị giá trị ra màn hình | 14. Vòng lặp |
| 7. Chuyển hướng nhập/xuất giá trị | 15. Hàm trong Shell Script |
| 8. Ống dẫn | 16. Bài tập |

3.15. Hàm trong Shell Script

- ❑ Hàm trong Shell Script là một tập hợp các câu lệnh được gom lại với nhau và có thể được gọi thực thi bất cứ khi nào cần thiết.

- ❑ **Cú pháp tạo hàm:**

```
function ten_ham() {  
    # Các lệnh bên trong hàm  
}
```

Hoặc

```
ten_ham() {  
    # Các lệnh bên trong hàm  
}
```

3.15. Hàm trong Shell Script (tt)

❑ Cú pháp gọi hàm (sử dụng hàm):

```
tên_hàm [Tham số 1] [Tham số 2] ... [Tham số n]
```

❑ Hàm không tham số:

Ví dụ 3.31:

```
#!/bin/bash
```

```
chao_mung() {  
    echo "Xin chào! Chào mừng đến với Bash scripting."  
}
```

```
# Gọi hàm  
chao_mung
```

3.15. Hàm trong Shell Script (tt)

❑ Hàm có tham số:

- \$1, \$2, ... là các tham số truyền vào hàm.
- Sử dụng local để khai báo biến cục bộ trong hàm.

Ví dụ 3.32:

```
#!/bin/bash
tong_hai_so() {
    local a=$1
    local b=$2
    local tong=$((a + b))
    echo "Tổng của $a và $b là: $tong"
}

# Gọi hàm với tham số
tong_hai_so 5 10
```

3.15. Hàm trong Shell Script (tt)

❑ Giá trị trả về của hàm:

- Hàm trong shell không trả về giá trị như các ngôn ngữ lập trình khác.
- Có thể dùng lệnh `return` để trả về giá trị của hàm với mã thoát (0-255).
- Giá trị này nằm trong biến `$?` và thường được sử dụng để kiểm tra xem hàm có chạy thành công hay không?
- Quy ước:
 - 0: Thành công
 - Khác 0: Lỗi hoặc thất bại

3.15. Hàm trong Shell Script (tt)

- ❑ **Lấy kết quả (mong muốn) từ hàm:** Dùng cú pháp `$ ( . . . )` để gán kết quả của một lệnh hoặc hàm vào biến toàn cục.

Ví dụ 3.33:

```
#!/bin/bash
tong_hai_so() {
    local a=$1
    local b=$2
    tong=$((a + b))
}

# Gọi hàm với tham số
tong_hai_so 5 10

#Hiển thị kết quả
echo "Tổng: $tong"
```

Nội dung giảng dạy

- | | |
|---|----------------------------|
| 1. Giới thiệu về Shell và Shell Script | 9. Toán tử trong Shell |
| 2. Tạo Shell Script | 10. Lệnh let |
| 3. Thực thi Shell Script | 11. Lệnh bc |
| 4. Biến trong Shell | 12. Lệnh awk |
| 5. Nhập giá trị từ bàn phím và lưu vào biến | 13. Cấu trúc điều kiện |
| 6. Hiển thị giá trị ra màn hình | 14. Vòng lặp |
| 7. Chuyển hướng nhập/xuất giá trị | 15. Hàm trong Shell Script |
| 8. Ống dẫn | 16. Bài tập |

3.16. Bài tập

- ❑ **Bài tập 1:** Viết chương trình `lscount.sh` cho phép liệt kê nội dung thư mục và hiện thêm tổng số tập tin, tổng số thư mục.
- ❑ **Bài tập 2:** Viết chương trình `fact.sh` cho phép tính giai thừa.
- ❑ **Bài tập 3:** Viết chương trình `ls.sh` cho phép liệt kê nội dung thư mục, với tập tin thì hiển thị tập tin và hiển thị thêm phần thông báo là tập tin, còn với thư mục thì hiển phần thông báo thư mục.
- ❑ **Bài tập 4:** Viết chương trình `fib.sh` cho phép in ra dãy n số fibonacci đầu tiên.

3.16. Bài tập (tt)

□ **Bài tập 5:** Viết chương trình in ra các thông tin sau:

- Hello <Tên người đăng nhập>
- Thư mục hiện hành
- Tập tin và thư mục, kể cả các thư mục ẩn trong thư mục hiện hành
- Ngày và giờ hiện tại

3.16. Bài tập (tt)

- ❑ **Bài tập 6:** Viết chương trình thực hiện các yêu cầu sau:
 - Yêu cầu nhập họ
 - Yêu cầu nhập tên
 - So sánh họ và tên nhập vào với họ và tên của bạn. Nếu trùng sẽ hiển thị thông báo ra màn hình.

3.16. Bài tập (tt)

- ❑ **Bài tập 7:** Lệnh `date +%H` sẽ in ra giờ hiện tại. Dùng lệnh này để in ra màn hình câu: hoặc Chào buổi sáng , hoặc chào buổi trưa, hoặc chào buổi tối, tùy thuộc vào thời điểm hiện tại.
- ❑ **Bài tập 8:** Viết chương trình in ra menu chọn lựa như sau:

1. Cafe đen
2. Cafe da
3. Cafe sữa
4. Cafe cao

Bạn chọn thức uống nào (1-4) ?

In ra tên thức uống mà khách hàng vừa chọn.

3.16. Bài tập (tt)

- ❑ **Bài tập 9:** Viết chương trình giả lập một máy tính gồm 4 phép tính. Chương trình in ra một menu như sau:

1. Cộng (+)
2. Trừ (-)
3. Nhân (*)
4. Chia (/)
5. Kết thúc (q)

Bạn chọn phép tính nào ?

Sau đó yêu cầu người dùng nhập tiếp 2 số nguyên, rồi thực hiện phép toán người dùng đã chọn, thông báo kết quả phép toán.

3.16. Bài tập (tt)

□ **Bài tập 10:** Viết chương trình thực hiện các yêu cầu sau:

- Chương trình nhận 2 tham số. Kiểm tra xem người dùng có nhập đúng 2 tham số không. Nếu không sẽ thoát.
- Tham số thứ nhất là một chuỗi str không có khoảng trắng, tham số thứ 2 là một số nguyên N. Dùng vòng lặp for để in ra chuỗi str N lần.